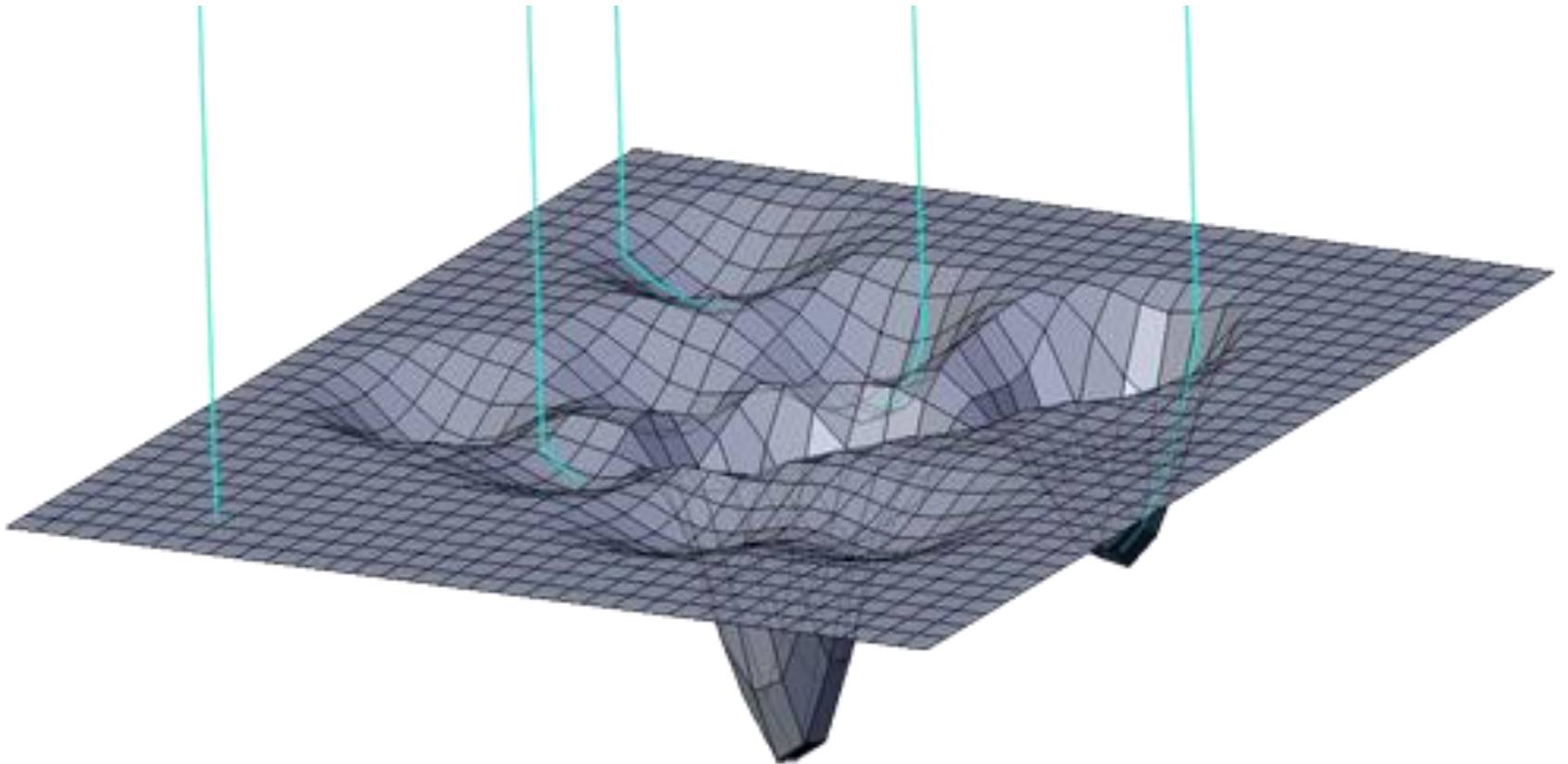


Lecture 10 – Distributions II & Model Choice



Today's Learning Outcomes

1. Be able to explain the system that the geometric, gamma, and beta distributions are useful to use as models
2. Be able to explain the idea of numerical optimization
3. Be able to apply a likelihood function and AIC calculation to a model

Probability Distributions

- Probability distributions are mathematical functions that describe the probability of an outcome
 - Used to represent how we think processes work and then test our expectations
- So far we have introduced....
 - Binomial (1 parameter: probability of success)
 - Normal (2 parameters: mean, standard deviation)
 - Uniform (2 parameters: minimum, maximum)
 - Exponential (1 parameter: decay rate)

Binomial Distribution in Python

- There are quite a few distributions built into Python **scipy** and they all have a common set of four functions

```
from scipy.stats import binom
```

- `binom.pmf`
 - **P**robability **M**ass **F**unction
- `binom.cdf`
 - **C**umulative **D**istribution **F**unction
- `binom.ppf`
 - **P**ercent **P**oint **F**unction
- `binom.rvs`
 - **R**andom **V**ariates



binom is the
abbreviation
for binomial
distribution

Normal Distribution (mean, sd)

```
from scipy.stats import norm
```

- Normal distribution is your simple bell curve
 - `norm.pdf(20, 20, 5)` = probability of getting the value 20 from a normal distribution with mean=20, sd=5
 - `norm.cdf(20, 20, 5)` = cumulative probability of getting a value equal to or less than 20
 - `norm.ppf(0.5, 20, 5)` = returns value where 50% of the cumulative probability is below
 - `norm.rvs(20, 5, size=20)` = generates 20 random numbers drawn from a normal distribution (mean=20, sd=5)

Probability Distributions

Let's quickly run through a few more...

- Geometric (1 parameter)
- Gamma (2 parameters)
- Beta (2 parameters)

Geometric Distribution

- Models the probability distribution of the Bernoulli trials needed to get one success
- Only parameter is probability of success (p) over some time interval
 - Ex. $p = 0.01$ for the probability of a 100-year flood per year

$$\Pr(x = k) = (1 - p)^{k-1}p$$

$$\text{Mean}(X) = \frac{1}{p}$$

$$\text{Variance} = \frac{1 - p}{p^2}$$

Example practice

For a 100-year flood ($p=0.01$), assumed it happened in 2000, we can apply the geometric distribution to model the probability of the flood occurred in a specific number of years later.

- What's their mean value of the interval?
- What's the variance and standard deviation?

Geometric Versus Binomial Distributions

- Both are discrete distributions (success/failure)
- The geometric distribution models probability of success of a given interval (often per year)
- The binomial distribution models probability of success per trial (time-independent)
 - Does not matter whether we wait 1 minute or 1 year between coin flips, but the time interval does matter to the probability of a flood occurring

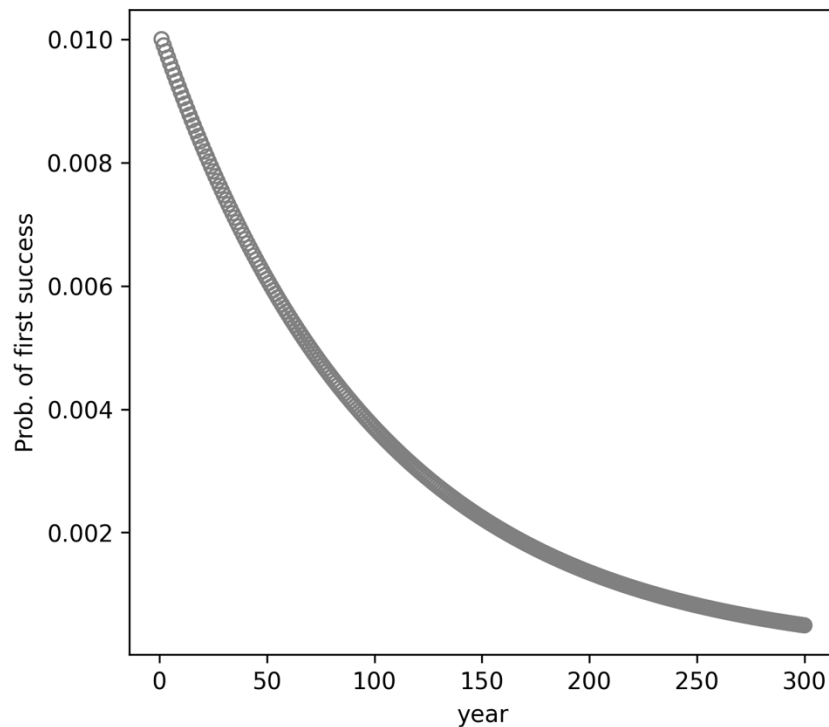
Geometric Distribution

```
from scipy.stats import geom
```

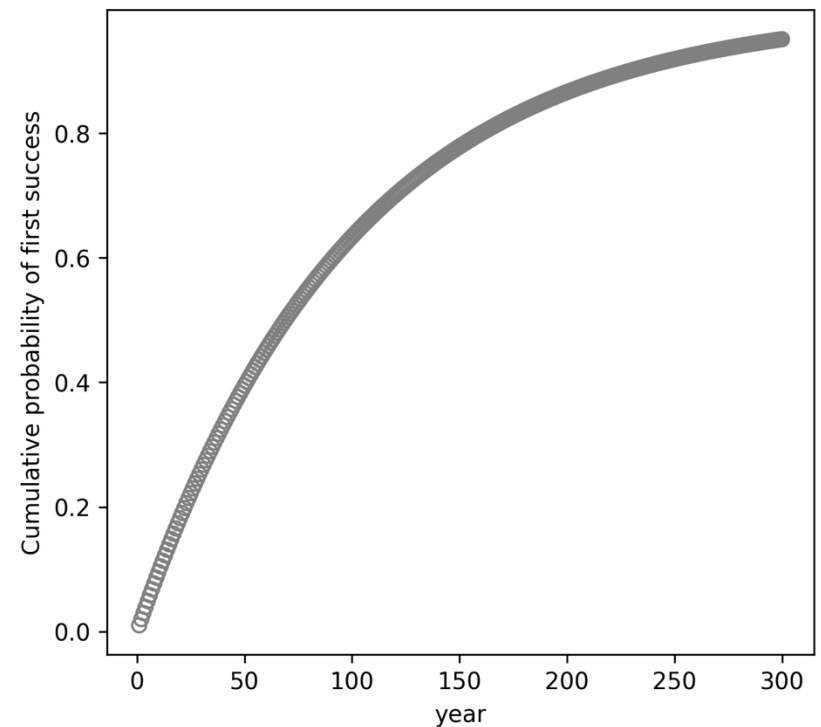
- `geom.pmf(x, p)` = probability of a wait time of duration x , given p per unit time chance
- `geom.cdf(x, p)` = probability of a wait time of x or less, given p
- `geom.ppf(q, p)` = quantile calculation
- `geom.rvs(n, p)` = generates n random values from a geometric distribution

What is the chance of experiencing a 100-year flood on a given river within the next 25 years?

Distribution of times between 100-year floods



Cumulative distribution of times between 100-year floods



```
geom.cdf(25, 0.01)  
[1] 0.2221786
```

Example Question: Flooding

The probability of seeing a 100-year flood in the next 25 years seems kind of high, at 22%

What's the median wait time for a 100-year flood (the 50-percentile value)?

```
geom.ppf(0.5, 0.01)
```

```
[1] 69
```



You might think the median would be 50...

```
geom.ppf(0.95, 0.01)
```

```
[1] 299
```



5% of the time you wait more than 299 years for a 100-year flood (heavily right-skewed distribution)

Geometric Distribution Uses

- Gamma distribution applies equally well to lots of processes geologists are interested in...
 - Floods
 - Earthquakes
 - Impact events
 - Eruptions
 - Coral bleaching events
- Anything where discrete events are occurring again and again through time

Exponential Vs Geometric

- The distribution of years between flood events (defined by some threshold value of stage or discharge volume) follows a geometric distribution (discrete variable: number of years between flood events)
- The distribution of discharge volume or stage will follow an exponential distribution (continuous variable: discharge volume or stage)
 - Chance of getting a value $\leq x$ from a random sample of measures [time frame problematic]

Gamma Distribution

- Can think of this as an expansion of the geometric distribution (continuous time dimension)
- Distribution of k successes where the probability of success follows a geometric distribution
 - Ex. We want to know how likely it is to get 10 100-year floods in 100 years
 - `gamma.cdf(T, a=k, scale=1/lam)`

$\text{lam} = 1/100$

$T = 100$

$k = 10$

Time interval
of interest

Number of “successes”
in the time interval

Probability of
“success” per year

Gamma Distribution

The Gamma Distribution always has two parameters, but they can be defined in three different ways

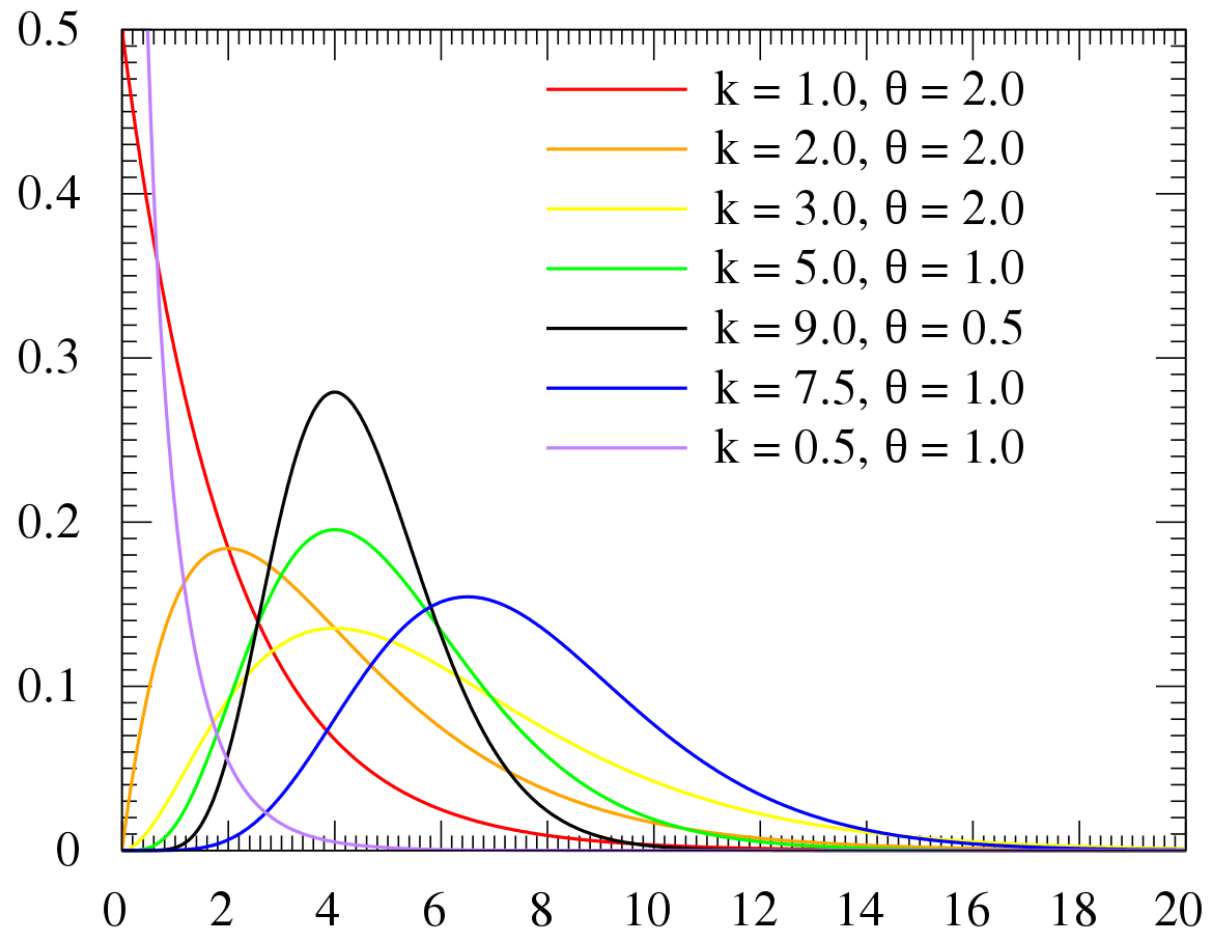
1. With a shape k (or a) and a scale parameter θ
2. With a shape k and a rate $b=1/\theta$
3. With a shape k and a mean value $= k/b$

It is not uncommon to see the same distribution described using different parameters in different fields

The mean of a gamma distribution is $\frac{k}{b}$, where k is the number of events, and b is the rate

If we are using
 $\theta = 1/b$,
the mean is then
 $k \cdot \theta$

The variance
is $k/b^2 = k \cdot \theta^2$



Gamma Distribution Example

- In a 100-year interval how likely is it that we get at least 2 100-year flood events

```
gamma.cdf(100, a=2, scale=100)  
[1] 0.2642411
```

- What's the probability of a once-in-a-lifetime event?

```
gamma.cdf(77, a=1, scale=77)  
[1] 0.6321206
```

- What's the probability of observing even a single once-in-a-millennium event in a lifetime?

```
gamma.cdf(77, a=1, scale=1000)  
[1] 0.07411015
```

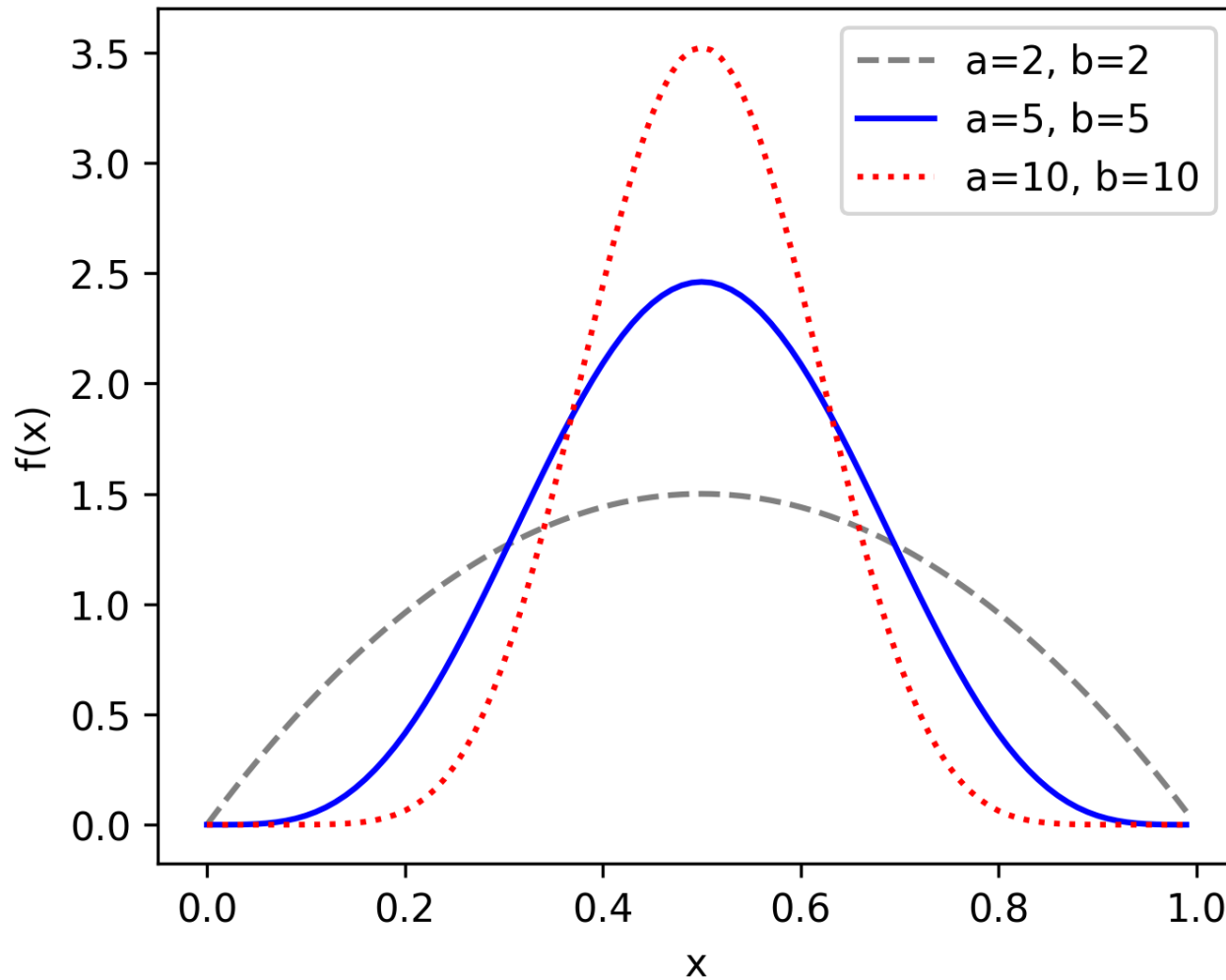
77 is the median lifespan in the United States

Beta Distribution

- Used to model percentages (0-100%)
 - Truncated bounds
 - `beta.pdf()`, `beta.cdf()`, `beta.ppf()`, `beta.rvs()`
- Two parameters (α and β , or in Python `a` and `b`)
 - `a=b` approximates a normal distribution, with standard deviation decreasing as `a` and `b` increases
 - `a>b` pulls distribution to the right
 - `b>a` pulls distribution to the left

When a and b are similar, mean is near 50%

Beta distributions with different parameters



Larger a and b values, produce smaller variance values when $a=b$,

if $a < b$, peak shifts to lower values,

if $a > b$ peak shifts to higher values

Beta Distribution

- Both a and b are *shape parameters*

- $mean = \frac{\alpha}{\alpha + \beta}$

- $variance = \frac{\alpha \times \beta}{(\alpha + \beta)^2 \times (\alpha + \beta + 1)}$

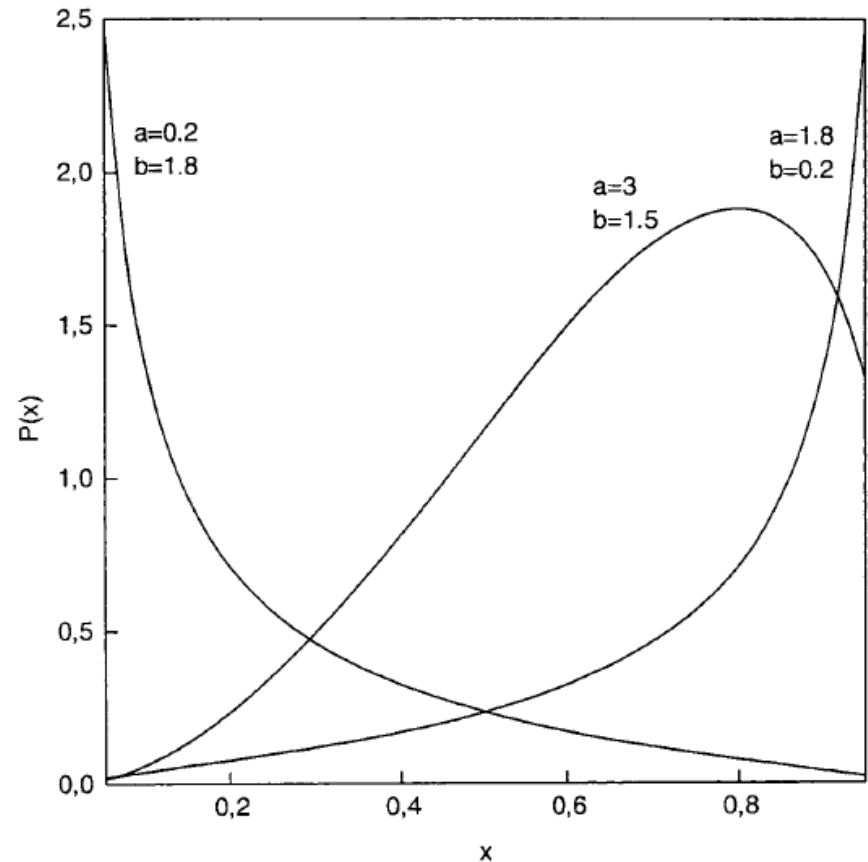


Fig. 1. Beta distribution with different parameter values.

Table 1
Parameters a and b for beta distribution model

	January	February	March	April	May	June	July	August	September	October	November	December
Petaling Jaya												
a	1.55	2.98	2.16	3.04	2.45	2.08	2.13	2.18	2.08	1.77	1.57	1.14
b	1.61	2.07	2.16	2.69	2.36	1.92	1.97	2.36	2.76	2.34	2.56	1.64
Subang												
a	1.55	2.91	2.18	2.73	2.34	1.51	1.99	2.00	2.14	1.73	1.75	1.28
b	1.49	1.94	2.02	2.14	2.08	1.24	1.70	2.00	2.51	2.20	2.51	1.63
Bayan Lepas												
a	2.20	3.94	2.87	2.43	1.81	1.71	2.02	1.55	1.23	1.47	1.54	1.35
b	0.99	1.31	1.55	1.43	1.54	1.19	1.52	1.55	1.56	1.80	1.42	0.80
Kota Bharu												
a	1.42	3.16	1.77	1.79	1.60	1.23	1.51	1.50	1.58	1.17	0.69	0.86
b	0.99	1.17	0.91	0.66	0.94	0.89	1.05	1.18	1.29	1.17	1.08	0.97

Models of sunlight patterns at three field stations, the data is collected daily over 10 years.

Which site and month has the highest mean sunlight percentage? Which locations are most consistent?

Optimization (numerical)

- Assuming we have a preferred model for a dataset how do we find the exact form of the model (i.e. parameter values) which best match the data?
 - Aka. optimizing the model fit
- In some cases, a precise analytical solution is possible but in many cases this is either impossible or computationally intensive
 - In those cases, we can use approaches which approximate the parameters of a model distribution instead

Optimization (numerical)

- To do this numerical approach we need an objective measure of error
- We then try lots of different parameter values and calculate the error for each parameter value(s)
 - The parameter values with the lowest error are the best fit to the data

If we start with the linear model $y = mx + b + \varepsilon$
 $\rightarrow \varepsilon = y - mx - b$

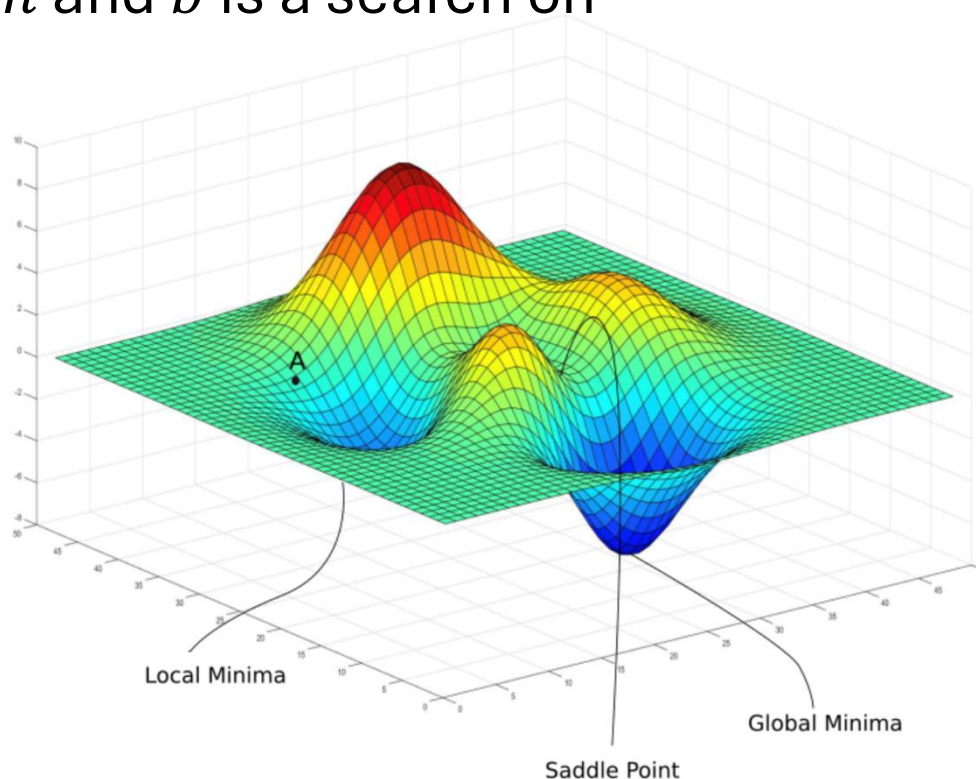
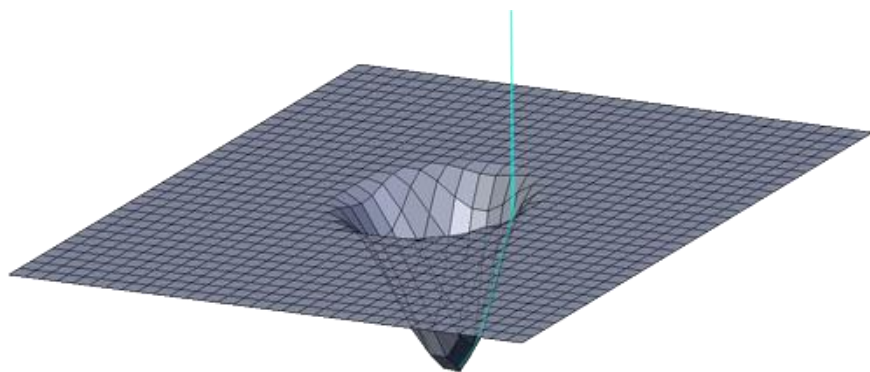
- Our objective function is the sum of squared residuals
 - $SSE = \sum_n \varepsilon_i^2$

Linear Model Optimization

- We can then calculate SSE for any combination of the parameters m and b in this model
- Our best estimate for m and b is the one which has the lowest SSE
- Picking m and b values at random to calculate SSE values for means we could run lots of calculations and still our best estimate is nowhere close to the real value
 - We need a guided way to search the parameter values and ideally a sensible starting point for m and b

If we think of the pair (m, b) as (x, y) coordinates, and the summed of squared results (the objective function) as a height (z) at any point (m, b) , then we have an *error landscape*.

Finding the best choice of m and b is a search on the error landscape to find the lowest point



Heuristic Search Algorithms

- We want to search the error landscape efficiently, most values are obviously very bad so we don't really want to bother checking them
 - Ex. checking a b value of 100 when the data crosses the y-axis near zero
- Heuristics algorithms are search methods that produce estimate efficiently, but cannot guarantee that the actual true answer has been located
 - Greedy algorithm ([Nelder-Mead](#))

`minimize ()` function in Python

- The Nelder Mead algorithm is a greedy downhill method, it always tries to move to a lower error (and thus can be easily trapped in local minima)
- The `minimize ()` function in Python will carry out a Nelder-Meade search to minimize an objective function we have created
 - $\epsilon = y - mx - b \rightarrow \text{error squared} = (y - mx - b)^2$



```
res = minimize(mse_loss_with_intercept,  
               theta0, args=(X, y), method='Nelder-Mead')
```

The loss function

Initial guess
of m and b

minimize () function in Python

```
res = minimize(mse_loss_with_intercept,  
               theta0, args=(X, y), method='Nelder-Mead')
```

```
def mse_loss_with_intercept(theta, X, y):
```

```
    """
```

```
    theta = [beta (d params), b (1 param)]
```

```
    """
```

```
    d = X.shape[1]
```

```
    beta = theta[:d]
```

beta represents slopes

```
    b = theta[d]
```

b represents an intercept

```
    resid = X @ beta + b - y
```

@ represents matrix multiplication

```
    return 0.5 / len(y) * np.dot(resid, resid)
```

np.dot also represents matrix multiplication

Not only works for simple linear regression, but also for multi-linear regression

```
import numpy as np
from scipy.optimize import minimize
```

Import packages

```
rng = np.random.default_rng(1)
n, d = 200, 1
X = rng.normal(size=(n, d))
beta_true = np.array([1.2])
b_true = 0.75
y = X @ beta_true + b_true + rng.normal(scale=0.2, size=n)
```

Define the true values

```
# initial guess for [beta0..., b0]
```

```
theta0 = np.zeros(d + 1)
```

Initialize the guess

```
def mes_loss_with_intercept():
```

```
    res = minimize(
        mse_loss_with_intercept,
        theta0,
        args=(X, y),
        method='Nelder-Mead',
        options={'maxiter': 5000, 'disp': True})
```

This loss function was defined in previous slide. We omitted it here due to limited space

Run optimization

```
beta_hat = res.x[:d]
b_hat = res.x[d]
print("Estimated beta:", beta_hat)
print("Estimated intercept:", b_hat)
print("True beta:", beta_true, "True intercept:", b_true)
```

Print output values

Likelihood & Model Choice

- Let's expand now and consider that we might not have strong prior idea of the model a dataset comes from or there are multiple explanatory variables for a single outcome
- How do we decide which of a set of candidate models is the best fit to the data?
 - Define an error metric (objective function), assuming a particular model for the error distribution calculate the summed probability of the observed errors
- This is a likelihood function... the probability of a set of observation assuming a specific model generated them

Likelihood function for errors in LR

Assume the classical linear regression model:

$$y_i = \beta_0 + \beta_1 x_i + \cdots + \beta_p x_{ip} + \varepsilon_i$$

with errors:

$$\varepsilon_i \sim i.i.d. N(0, \sigma^2).$$

i.i.d. Independent and
Identically Distributed

$N(0, \sigma^2)$. Normal distribution
with mean =0, variance= σ^2

Each error term has density:

$$f(\varepsilon_i) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{\varepsilon_i^2}{2\sigma^2}\right)$$

Likelihood function for errors in LR

Because the errors are *independent*

$$L(\beta, \sigma^2 \mid y) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - X_i^\top \beta)^2}{2\sigma^2}\right).$$

This is the likelihood function in its raw form.

Likelihood function for errors in LR

Take logs (simplifies products $\rightarrow$ sums), and plug the MLE for σ^2 in linear regression ($\hat{\sigma}^2 = \frac{RSS}{n}$) into the raw likelihood function:

$$\ell(\hat{\beta}, \hat{\sigma}^2) = -\frac{n}{2} \left[\log(2\pi) + \log\left(\frac{RSS}{n}\right) + 1 \right].$$

Log-likelihood and AIC

We can make three additional adjustments to our likelihood function to arrive at a value called the Akaike's Information Criterion (AIC)...

1. Log-transform the likelihood function
 - Converts products to sums and stabilizes computation
2. Evaluate the log-likelihood at its maximum
 - Gives $\text{Ln}(\hat{L})$: how well the model fits the data
3. Apply Akaike's tradeoff between fit and complexity
 - Penalize complexity: $+2k$
 - Reward goodness of fit: $-2\text{Ln}(\hat{L})$

The best-supported model is the one with the **lowest AIC score**

Akaike's Information Criterion

$$AIC = 2k - 2 \ln(L)$$

where L represents the likelihood function; $\ln$ is natural log, in python `np.log()` is by default the natural log

We can then give relative weights to the models by comparing their deviances in AIC relative to the best model

- AIC weight = $e^{(-1/2(\text{Model AIC} - \text{Lowest Model AIC}))}$

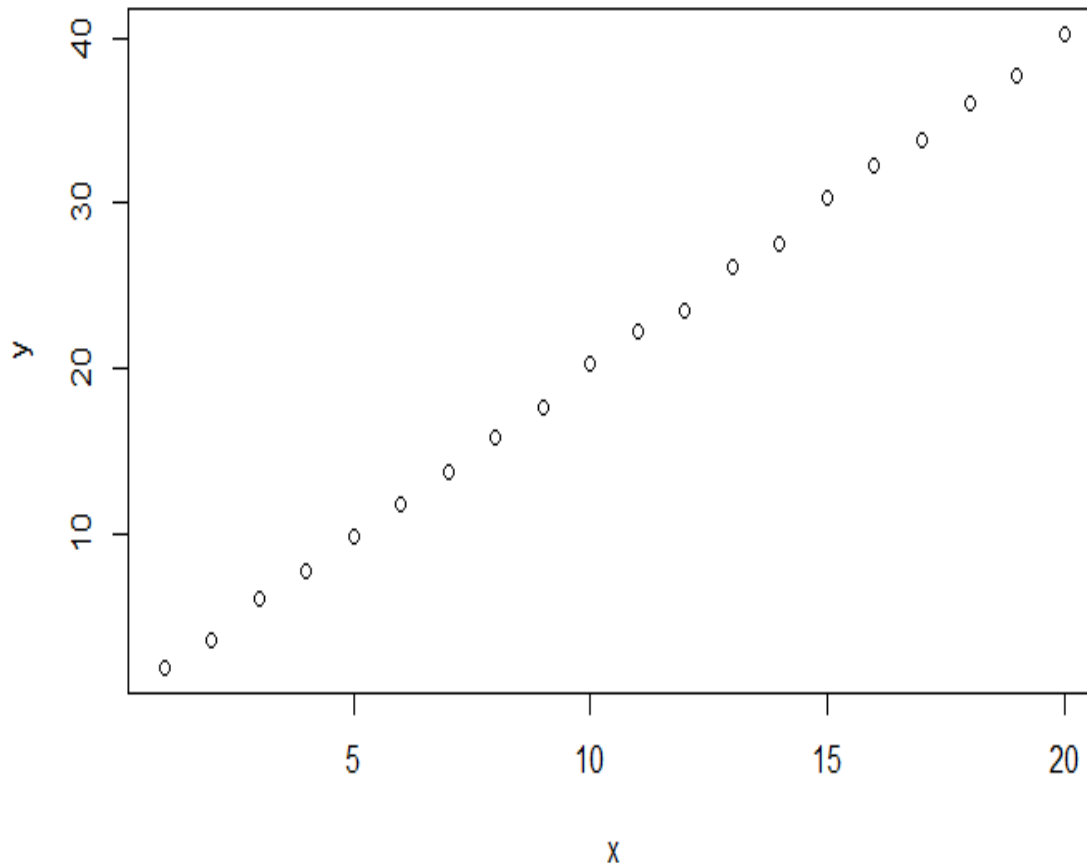


Best model AIC weight will always = 1

Akaike Weights

- The final step in model comparison is to try and get some idea of which models to consider
- Relative AIC weight = $(\text{AIC Model Weight}) / \text{sum}(\text{all AIC Model Weights})$
- Relative AIC weight will always sum to 1 and gives the probability of support for each model
 - Typically we consider a set of models whose relative AIC weights sum to 0.95 to be a set of models we should consider as possible explanations of our data

Example Use of AIC



How could we model this regression?

$n=20$ x-y pairs

How complex a polynomial should we use to describe this data?

Candidate models

1.) $y = m$; no slope, just a mean value, $k=1$

2.) $y = mx + b$; linear regression, slope and intercept, $k=2$

3.) $y = ax^2 + bx + c$; 2nd order polynomial, $k=3$

4.) $y = ax^3 + bx^2 + cx + d$; 3rd order polynomial, $k=4$

Let's create log-likelihood functions for each model

```

def gaussian_nll_from_resid(resid):
    # make sure that resid do not contain any NaN or Inf values
    r = resid
    rss = np.sum(r**2)
    sigma2_hat = rss / n # MLE
    # -log L = n/2 * [ log(2π) + log(sigma2_hat) + 1 ]
    return 0.5 * n * (np.log(2*np.pi) + np.log(sigma2_hat) + 1.0)

# y = m
def nll_a(theta, x, y):
    (m,) = theta
    resid = y - m
    return gaussian_nll_from_resid(resid)

```


AIC versus BIC

- BIC stands for Bayesian Information Criterion.

$$\text{AIC} = 2k - 2 \ln(L)$$

$$\text{BIC} = 2k \cdot \ln(n) - 2 \ln(L)$$

- AIC is best for predictive performance, while BIC heavily penalizes complex models, favoring simpler, more interpretable models

Today's Learning Outcomes

1. Be able to explain the system that the geometric, gamma, and beta distributions are useful to use as models
2. Be able to explain the idea of numerical optimization
3. Be able to apply a likelihood function and AIC calculation to a model